

コンパイラ (8)

2026/10/27

コード最適化

コード最適化

実行の意味を変えずに実行を高速化すること。

実行の意味：入力と出力

最適化の対象によって入力と出力は変わる。

プログラム解析が必要

- ▶ 機械独立なコード最適化
＝3 番地コードのレベルで実行を効率化
- ▶ 機械依存なコード最適化
＝対象マシンのアーキテクチャに依存
(VLIW、マルチコアなど)

基本ブロック

最後以外分岐のない文の並び

```
prod := 0;  
i := 0;  
while (i < 20) begin  
    prod := prod + a[i]*b[i];  
    i := i + 1  
end;
```

```
prod = 0  
i = 0
```

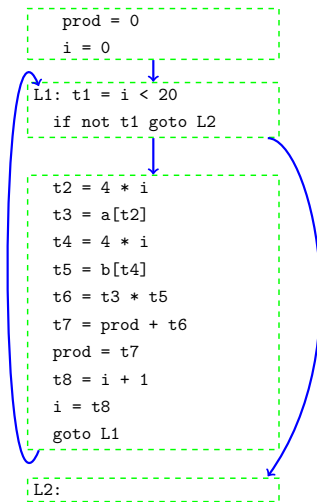
```
L1: t1 = i < 20  
    if not t1 goto L2
```

```
t2 = 4 * i  
t3 = a[t2]  
t4 = 4 * i  
t5 = b[t4]  
t6 = t3 * t5  
t7 = prod + t6  
prod = t7  
t8 = i + 1  
i = t8  
goto L1
```

基本ブロック

最後以外分岐のない文の並び

```
prod := 0;  
i := 0;  
while (i < 20) begin  
    prod := prod + a[i]*b[i];  
    i := i + 1  
end;
```



局所最適化

基本ブロック内だけを見て最適化すること

覗き穴最適化 (Peep-hole optimization)

- ▶ 定数伝播 (constant propagation)
定数の計算、参照を定数そのものでおきかえる
- ▶ 共通部分式の削除 (common subexpression elimination)
共通の部分式を抽出して結果を一時変数に保存するようにして、二回目以降計算しない。
- ▶ コピー伝播 (copy propagation)
余分な変数の仲介を減らす。
データ依存性を減らして命令の並列発行が可能になる場合がある。
- ▶ 不要コードの除去 (deadcode elimination)
使われない変数に関する計算を消去する。
- ▶ 代数的性質の利用
演算子が持つ代数的性質 (可換則、結合則、分配則、計算のためのコストなど)

DAG による局所最適化

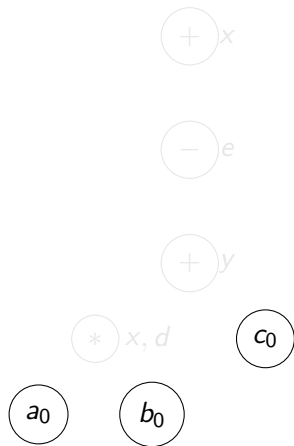
DAG=Directed Asyclic Graph

閉路 (ループ) をもたない有効グラフ

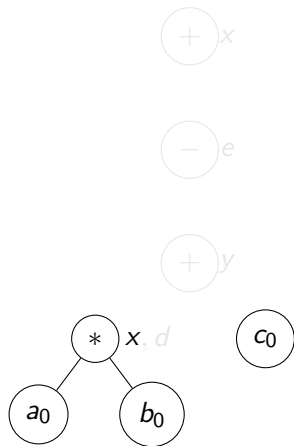
基本ブロック内のデータ依存性を一意的に表現

DAG の構成アルゴリズム 6.6

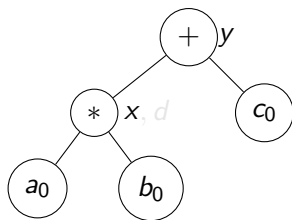
DAG の構成

 $x = a * b$ $y = x + c$ $d = a * b$ $e = d - c$ $x = d + e$

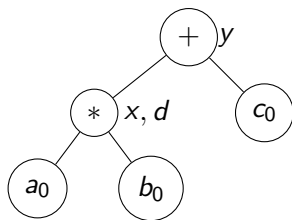
DAG の構成



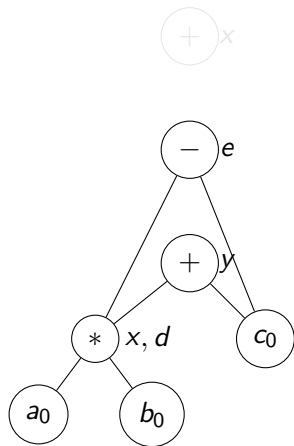
DAG の構成

 $x = a * b$ $y = x + c$

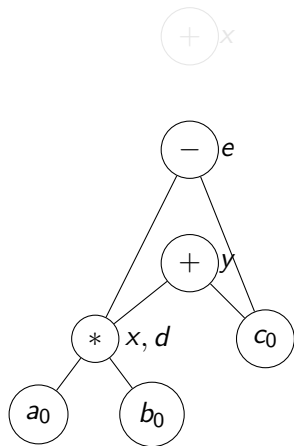
DAG の構成

 $x = a * b$ $y = x + c$ $d = a * b$

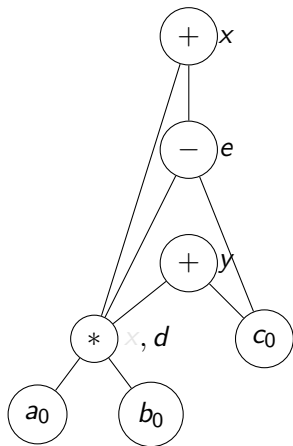
DAG の構成

 $x = a * b$ $y = x + c$ $d = a * b$ $e = d - c$

DAG の構成

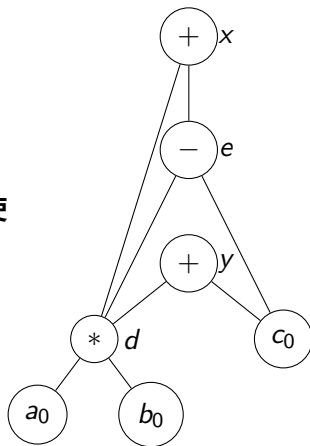
 $x = a * b$ $y = x + c$ $d = a * b$ $e = d - c$ $\Rightarrow$ $x = a * b$ $y = x + c$ $d = x$ $e = d - c$

DAG の構成

 $x = a * b$ $y = x + c$ $d = a * b$ $e = d - c$ $x = d + e$ $d = a * b$ $y = d + c$ $e = d - c$ $x = d + e$

不要コードの消去

これ以降、 x しか使用しないとすると

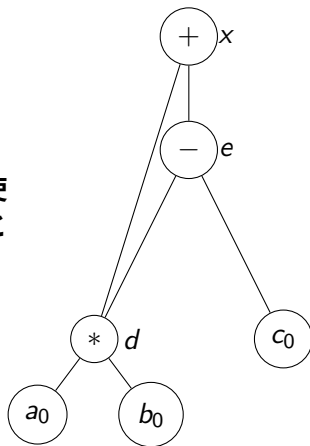


$d = a * b$
 $y = d + c$
 $e = d - c$
 $x = d + e$

不要な変数を上からたどって消していく。
 e, d は直接使わないが、 x の計算に必要なので消去できない。

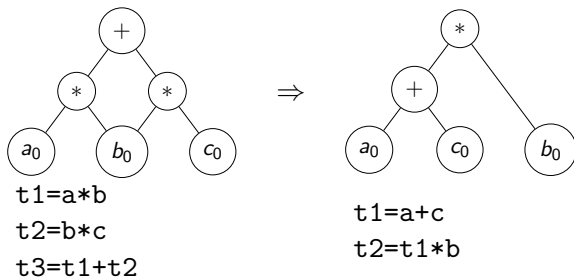
不要コードの消去

これ以降、 x しか使
用しないとすると
 $y = d + c$ は消去

 $d = a * b$ $e = d - c$ $x = d + e$

不要な変数を上からたどって消していく。
 e, d は直接使わないが、 x の計算に必要なので消去できない。

代数的性質



分配則=DAG 上でのパターンマッチング